

ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

OBJECT-ORIENTED PROGRAMMING

PAPER 1 · PRACTICE A

MARKS: 100 **TIME: 3 hours**

BEng Mechatronics and Robotics Engineering, 2nd year final examination (C++): object-oriented concepts, classes, inheritance and polymorphism, operator overloading, memory management, basic data structures (linked lists, stacks, queues), exceptions and templates.

Written by Academic Central for practice, following object-oriented programming modules at South African universities (e.g. University of Pretoria COS 110 Program Design: inheritance, polymorphism, operator overloading, memory management, encapsulation, lists, stacks and queues), in C++. It is not a university examination paper.

INSTRUCTIONS AND INFORMATION

1. This question paper consists of SIX questions. Answer ALL the questions.
2. All code must be written in standard C++ (C++11 or later). Assume the necessary headers are included and that using namespace std; is in effect.
3. Small syntax mistakes that do not change the logic will not be penalised in QUESTIONS 3, 5 and 6.
4. No computers or calculators may be used.
5. Number your answers exactly as the questions are numbered.

QUESTION 1: OBJECT-ORIENTED CONCEPTS

1.1 Explain each of the following, with a short example from a robot program: encapsulation, inheritance, polymorphism and abstraction. (8)

1.2 What is the difference between a class and an object? (2)

1.3 Explain the difference between static (early) binding and dynamic (late) binding of member functions in C++. Which keyword makes dynamic binding possible? (3)

1.4 How is an interface (abstract class) written in C++? (2)

[15]

QUESTION 2: TRACING CODE

Give the exact output of each program.

2.1 Constructors and destructors: (4)

```
class A { public: A() { cout << "A"; } ~A() { cout << "~A"; } };
class B : public A { public: B() { cout << "B"; } ~B() { cout << "~B"; } };
int main() { B b; cout << "|"; }
```

2.2 Virtual and non-virtual functions: (4)

```
class Sensor {
public:
    virtual string read() const { return "sensor"; }
    string id() const { return "S"; }
};
class Lidar : public Sensor {
public:
    string read() const override { return "lidar"; }
    string id() const { return "L"; }
};
int main() {
    Lidar l;
    Sensor* p = &l;
    cout << p->read() << " " << p->id() << endl;
}
```

2.3 Pass by value and by reference: (3)

```
void f(int a, int& b) { a += 5; b += 5; }
int main() { int x = 1, y = 1; f(x, y); cout << x << " " << y; }
```

2.4 A static member: (3)

```
class Robot {
    static int count;
public:
    Robot() { count++; }
    static int total() { return count; }
};
int Robot::count = 0;
int main() {
    Robot a, b;
    { Robot c; }
    Robot d[2];
    cout << Robot::total();
}
```

2.5 Operator overloading: (3)

```
struct Vec {
    double x, y;
    Vec operator+(const Vec& o) const { return {x + o.x, y + o.y}; }
    Vec operator*(double k) const { return {x * k, y * k}; }
};
int main() {
    Vec a{1, 2}, b{3, -1};
    Vec c = (a + b) * 2;
    cout << c.x << "," << c.y;
}
```

2.6 A stack: (3)

```
stack<int> s;
s.push(1); s.push(2); s.push(3);
s.pop();
s.push(4);
cout << s.top() << " " << s.size();
```

[20]

QUESTION 3: CLASS DESIGN

A robot arm has different actuators. Every actuator has a name and can be told to move by a number of degrees. A servo motor stores its angle and may only be between 0° and 180°. A stepper motor has 200 steps per revolution and stores its position as a number of steps.

3.1 Draw a UML class diagram for the abstract class Actuator and the classes ServoMotor and StepperMotor. Show attributes, methods, visibility (+ / − / #) and the inheritance arrows. (6)

3.2 Write the abstract class Actuator with a protected name, a constructor, a getName() method, a pure virtual method void move(double degrees) and a virtual destructor. (5)

3.3 Write the class ServoMotor. Its move() method adds degrees to the angle but keeps the angle between 0 and 180. (6)

3.4 Write the class StepperMotor. Its move() method converts degrees to steps (rounded to the nearest step) and adds them to the position. (5)

3.5 Write a main() that stores one ServoMotor and one StepperMotor in a vector of Actuator pointers, moves every actuator by 90° in a loop, and then frees the memory. (3)

[25]

QUESTION 4: MEMORY MANAGEMENT

4.1 Explain the difference between memory on the stack and memory on the heap. (2)

4.2 The code below contains THREE memory errors. Identify each one, explain why it is a problem and how to fix it. (6)

```
int* makeArray() {
    int data[10];
    return data;          // line A
}
int main() {
    int* p = new int[5];
    p = new int[10];      // line B
    int* q = p;
    delete[] p;
    delete[] q;           // line C
}
```

4.3 A class holds a raw pointer to an array on the heap. Explain the difference between a shallow copy and a deep copy, and why such a class needs a copy constructor, a copy assignment operator and a destructor (the rule of three). (4)

4.4 How does std::unique_ptr help to prevent memory leaks? (3)

[15]

QUESTION 5: DATA STRUCTURES

5.1 Write a struct Node for a singly linked list of integers, and a function void pushFront(Node*& head, int value) that adds a new node at the front of the list. (5)

5.2 Explain the difference between a stack and a queue, and give ONE use of each in a robot control program. (4)

5.3 Write a class SensorBuffer that keeps the last 5 readings (doubles) in a fixed array used as a circular buffer. It needs a method add(double r) that overwrites the oldest reading when the buffer is full, and a method average() that returns the average of the readings stored so far. (6)

[15]

QUESTION 6: EXCEPTIONS AND TEMPLATES

6.1 Write a function template largest(a, b) that returns the larger of two values of any type that supports >. (3)

6.2 Write a function setSpeed(int s) that throws std::out_of_range if s is not from 0 to 100, and a main() that calls it with 150 and catches the exception, printing its message. (4)

6.3 Why should a member function that does not change an object be declared const? Give an example. (3)

[10]

TOTAL: 100

END OF QUESTION PAPER